

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1710

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

**Duration : 1 Hour
Total Marks:50**

Annexure A

Reverse Engineering

Code of Conduct:

I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: POOJASREE B

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Student 23: POOJASREE B | Register Number: 192411091

Reverse Engineering a Ride-Hailing Dispatch System

A ride-hailing app matches riders to nearby drivers within seconds, even during peak demand. By observing its outputs and behavior, reconstruct the distributed AI pipeline using PySpark RDDs. Explain how location-data processing, demand forecasting, and agent-based driver-matching mechanisms are likely implemented.

Reverse Engineering a Ride-Hailing Dispatch System

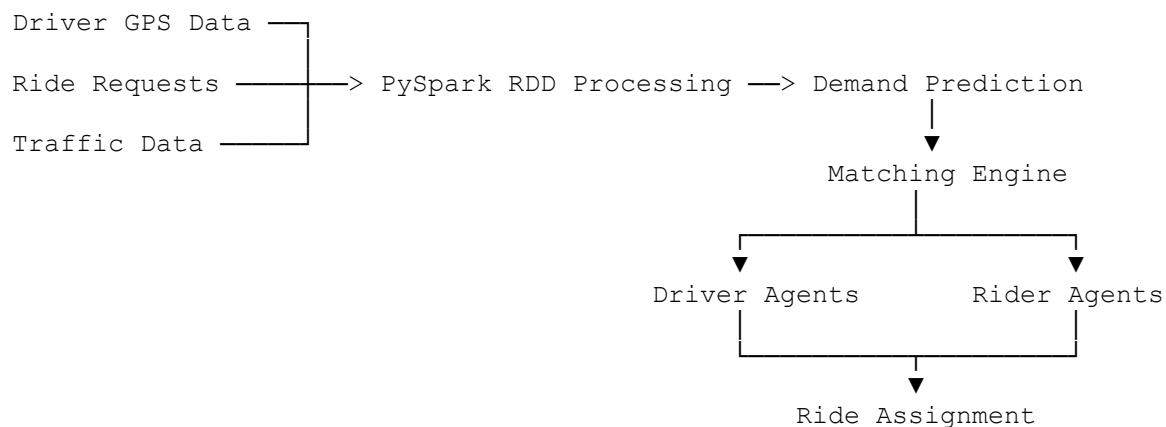
1. Introduction

A ride-hailing application must match riders with suitable nearby drivers within seconds. During peak hours, the system processes massive amounts of real-time data, including driver locations, ride requests, traffic conditions, and historical demand patterns.

A likely architecture combines **PySpark RDDs for distributed data processing**, **AI-based demand forecasting**, and **agent-based matching mechanisms** to provide fast and efficient ride allocation.

2. Reconstructed Distributed AI Pipeline

A possible pipeline can be represented as:



The system continuously receives data streams from thousands of drivers and riders and distributes the computation across multiple machines.

3. Location Data Processing Using PySpark RDDs

Each driver continuously sends GPS coordinates such as:

```
(driver_id, latitude, longitude, availability, timestamp)
```

Ride requests may contain:

```
(rider_id, pickup_latitude, pickup_longitude, destination, timestamp)
```

These records can be stored and processed as PySpark RDDs.

a) `map()` Transformation

The `map()` operation converts raw GPS data into structured records.

```
driver_rdd = raw_data.map(lambda x: x.split(","))
```

It can also transform GPS coordinates into geographical zones.

```
zone_rdd = driver_rdd.map(  
    lambda x: (get_zone(x[1], x[2]), x)  
)
```

This helps group drivers according to their location.

b) `filter()` Transformation

The system filters only active and available drivers.

```
available_drivers = driver_rdd.filter(  
    lambda x: x[3] == "AVAILABLE"  
)
```

This prevents unavailable or busy drivers from entering the matching process.

c) `reduceByKey()` Transformation

Drivers and ride requests can be grouped by geographic zones.

```
zone_count = available_drivers \  
    .map(lambda x: (x[0], 1)) \  
    .reduceByKey(lambda a, b: a + b)
```

This calculates the number of available drivers in each zone.

Similarly, ride requests can be aggregated to identify high-demand areas.

d) `join()` Transformation

A join operation can combine ride requests with drivers located in the same geographical zone.

```
matches = rider_zone_rdd.join(driver_zone_rdd)
```

The system can then calculate the distance between riders and candidate drivers.

This distributed processing allows matching to happen simultaneously across many geographic regions.

4. Demand Forecasting

Ride-hailing platforms need to predict future demand before requests actually occur.

Historical data may include:

- Time of day
- Day of week
- Location
- Weather
- Traffic conditions
- Local events
- Previous ride requests

The data can be aggregated using RDD transformations:

```
demand_data = rides_rdd.map(  
    lambda x: ((x.zone, x.hour), 1)  
) .reduceByKey(lambda a, b: a + b)
```

The processed data can then be supplied to a machine learning model.

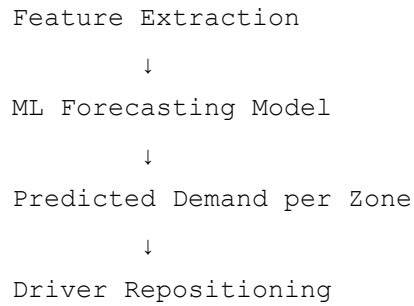
Possible Forecasting Pipeline

Historical Ride Data

↓

RDD Cleaning and Aggregation

↓



The AI model predicts zones where ride demand is likely to increase.

For example:

Zone	Current Drivers	Predicted Requests
A	50	120
B	80	40
C	30	90

The system can identify Zones A and C as high-demand areas and encourage idle drivers to move toward those locations.

5. Agent-Based Driver Matching

The dispatch system can be viewed as a **multi-agent system**.

Rider Agent

Each rider request acts as an agent with goals such as:

- Find a nearby driver.
- Minimize waiting time.
- Minimize ride cost.

Driver Agent

Each driver acts as an autonomous agent with information about:

- Current location.

- Availability.
- Vehicle type.
- Estimated arrival time.
- Current workload.

Dispatch Agent

A central or distributed dispatch agent coordinates the matching process.

Its objective is to optimize:

- Rider waiting time.
- Driver utilization.
- Travel distance.
- Ride acceptance probability.
- Overall system efficiency.

6. Driver Matching Process

A likely matching algorithm follows these steps:

Step 1: Receive Ride Request

A rider submits a pickup request.

Rider → Pickup Location → Dispatch System

Step 2: Identify Nearby Drivers

The location-processing system searches the rider's geographical zone and neighboring zones.

Step 3: Filter Eligible Drivers

Drivers are filtered based on:

- Availability

- Vehicle type
- Distance
- Driver status

Step 4: Calculate Matching Score

Each candidate driver may receive a score.

```
Match Score =  
w1 × Distance Score +  
w2 × ETA Score +  
w3 × Acceptance Probability +  
w4 × Driver Availability
```

The driver with the highest overall score is selected.

Step 5: Send Ride Offer

The selected driver receives the ride request.

```
Rider Request  
    ↓  
Nearby Drivers  
    ↓  
Score Calculation  
    ↓  
Best Driver Selected  
    ↓  
Ride Offer
```

If the driver rejects the request, the next-best candidate is selected.

7. Handling Peak Demand

During peak demand, the number of ride requests may exceed the number of available drivers.

The system can detect this using distributed RDD aggregation.

```
supply = drivers.map(  
    lambda x: (x.zone, 1)  
) .reduceByKey(lambda a, b: a + b)
```

```
demand = requests.map(  
    lambda x: (x.zone, 1)  
) .reduceByKey(lambda a, b: a + b)
```

The system compares supply and demand:

```
Demand > Available Drivers  
    ↓  
High-Demand Zone Detected  
    ↓  
AI Dispatch Optimization  
    ↓  
Driver Repositioning  
    ↓  
Dynamic Pricing / Incentives
```

This enables the platform to react quickly to changing conditions.

8. Role of PySpark RDDs

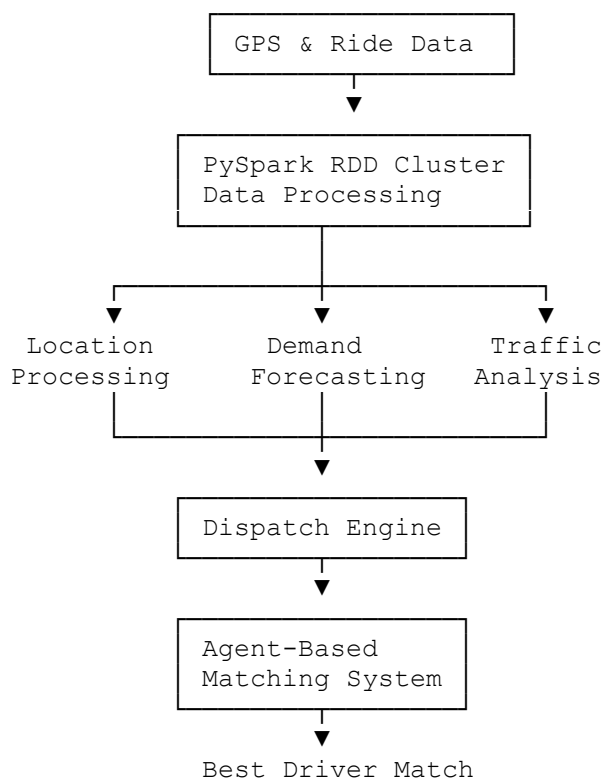
PySpark RDDs support the system through distributed operations:

RDD Operation	Role in Ride-Hailing System
<code>map()</code>	Transform GPS and request data
<code>filter()</code>	Select available drivers
<code>flatMap()</code>	Process multiple location events
<code>reduceByKey()</code>	Count drivers and requests per zone
<code>groupByKey()</code>	Group data by geographical area
<code>join()</code>	Combine riders with nearby drivers
<code>sortBy()</code>	Rank candidate drivers
<code>cache()</code>	Store frequently used location data

RDDs provide:

- Parallel processing
- Fault tolerance
- Scalability
- Distributed computation
- Faster processing of large datasets

9. Overall Reconstructed System



10. Conclusion

A scalable ride-hailing dispatch system can be reconstructed as a combination of **distributed PySpark RDD processing, AI-based demand forecasting, and multi-agent decision-making**. RDD transformations process massive volumes of GPS and ride-request data in parallel, while forecasting models predict future demand across geographic zones. Intelligent rider, driver, and dispatch agents then coordinate to select the most suitable driver based on distance, ETA, availability, and predicted system demand. This architecture enables fast, scalable, and adaptive ride matching even during peak demand.